

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : PADITYA

Reg. No.: 11249A301

Class : II B.E. (CSE)

Course Code: BCSF244AI2

Course Name: DATA SCIENCE TOOLKIT

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by
Mr/Ms. P ADITYA

with Reg. No 11249A301
academic year 2025 – 2026.

of II-B.E.(CSE) during the

Station: Enathur.

Date: 25-4-26

Staff-in-charge

Head of the Department

Submitted for the Practical examination held on

Examiner-1

Examiner-2

INDEX

SL.NO	Program Name	Date	Page no	Signature
1	Introduction to python libraries	29/1/26	1-10	
a	Numpy	29/1	1-10	
b	Pandas	29/1	1-10	
c	Matplotlib	5/2/26	1-10	
d	Scikit-learn	5/26	1-10	
2	Perform data exploration and preprocessing python	13/2/26	11-12	
3	Implement Regularized Linear Regression	19/2/26	13	
4	Implement Naïve Bayes Classifier	5/3/26	14-15	
5	Implement Regularized Logistic Regression	5/3/26	16-20	
6	Build models using ensembling techniques	12/3/26	21-22	
7	Build models using decision tree	12/3/26	23	
8	Support vector machines with different Kernels	2/4/26	24	
9	Implement K-Nearest Neighbours Classification	2/4/26	25-26	

10	K-Means clustering with PCA and elbow method	2/4/26	27-29	
-	-		-	-

Ex No:1

Date:05-02-26

Introduction to Python Libraries for Data Science

Aim: To learn basic Python libraries for Data Science (NumPy, Pandas, Matplotlib, Scikit learn).

Program:

```

import numpy as np
a = np.array([1,2,3,4,5])
b= np.arange(1,10,2)
c= np.linspace(0,1,5) //11249A301
d= np.zeros((2,3)) //P.ADITYA
e=np.ones((2,2))
f=np.eye(3)
g=np.random.randint(1,100,size=(3,3))
print("Array a:",a)
print("linspace c:",c)
print("zeros d:\n",d)
print("ones e:\n",e)
print("identity f:\n",f)
print("random g:\n",g)

*** Array a: [1 2 3 4 5]
linspace c: [0.  0.25 0.5  0.75 1. ]
zeros d:
[[0. 0. 0.]
 [0. 0. 0.]]
ones e:
[[1. 1.]
 [1. 1.]]
identity f:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
random g:
[[98 90 89]
 [ 5 80 74]
 [45 21 25]]

```

INPUT:

```
[ ] import numpy as np
    print("\nshape:",g.shape)           //11249A301
    print("dimensions:",g.ndim)        //P.ADITYA
    print("size:",g.size)
    print("data type:",g.dtype)
```

shape: (3, 3)
dimensions: 2
size: 9
data type: int64

```
[ ] import numpy as np
    reshaped=g.reshape(9,1)           //11249A301
    print("\nreshaped array:",reshaped) //P.ADITYA
    print("Transpose:\n",g.T)
    print("Flattened:",g.flatten())
```

reshaped array: [[78]
[97]
[29]
[71]
[55]
[35]
[2]
[72]
[98]]
Transpose:
[[78 71 2]
[97 55 72]
[29 35 98]]
Flattened: [78 97 29 71 55 35 2 72 98]

INPUT:

```
import numpy as np
print("\nAdd:",np.add(a,2))           //11249A301
print("multiply:",np.sqrt(a))        //P.ADITYA
print("power:",np.power(a,2))
print("absolute:",np.abs([-1,-2,3]))
```

```
Add: [3 4 5 6 7]
multiply: [1.         1.41421356 1.73205081 2.         2.23606798]
power: [ 1  4  9 16 25]
absolute: [1 2 3]
```

```
▶ print("\nmean:",np.mean(a))
print("median:",np.median(a))        //11249A301
print("standard deviation:",np.std(a)) //P.ADITYA
print("variance:",np.var(a))
print("minimum:",np.min(a))
print("maximum:",np.max(a))
print("sum:",np.sum(a))
print("percentile (50):",np.percentile(a,50))
```

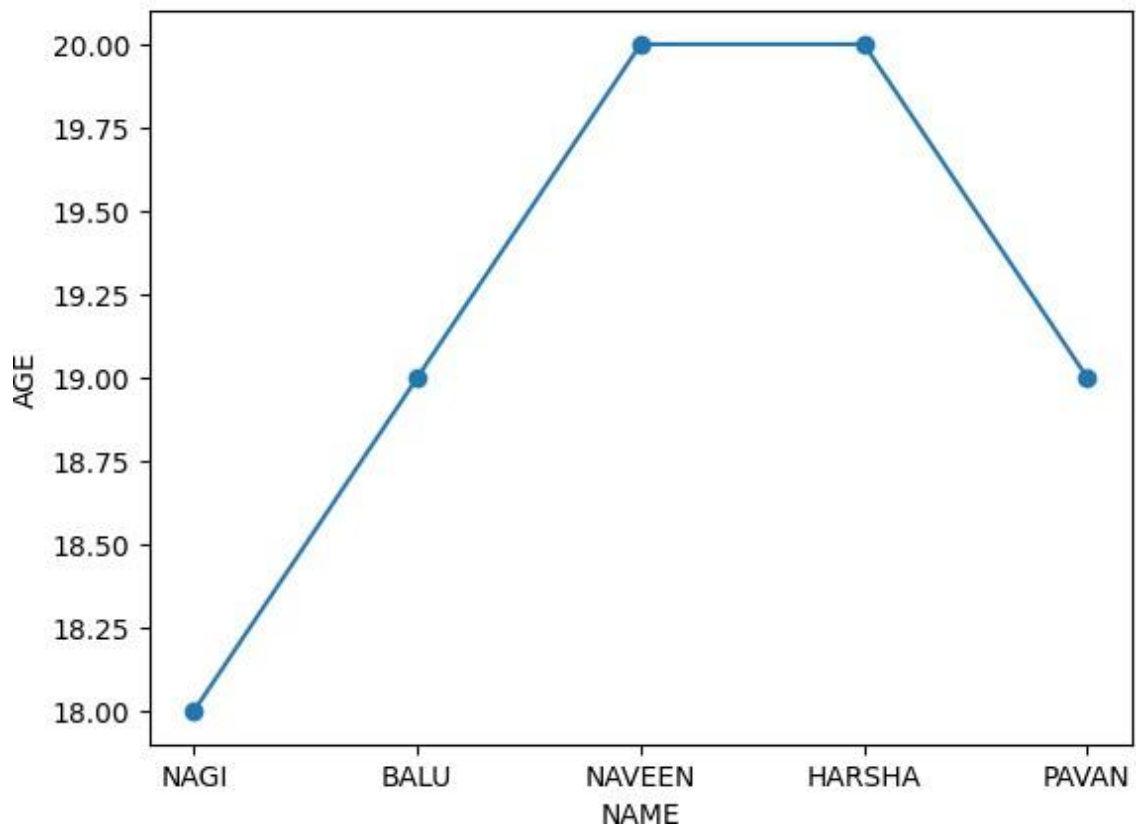
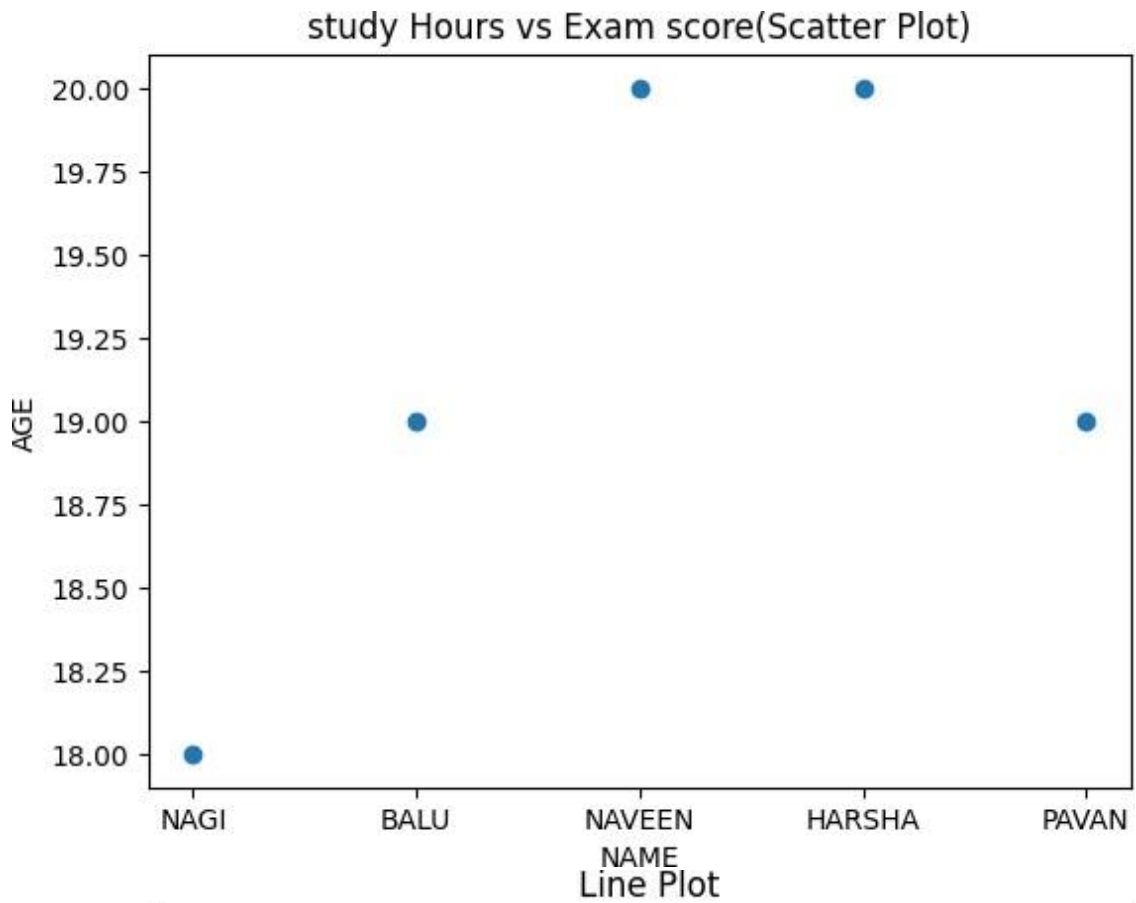
```
...
mean: 3.0
median: 3.0
standard deviation: 1.4142135623730951
variance: 2.0
minimum: 1
maximum: 5
sum: 15
percentile (50): 3.0
```

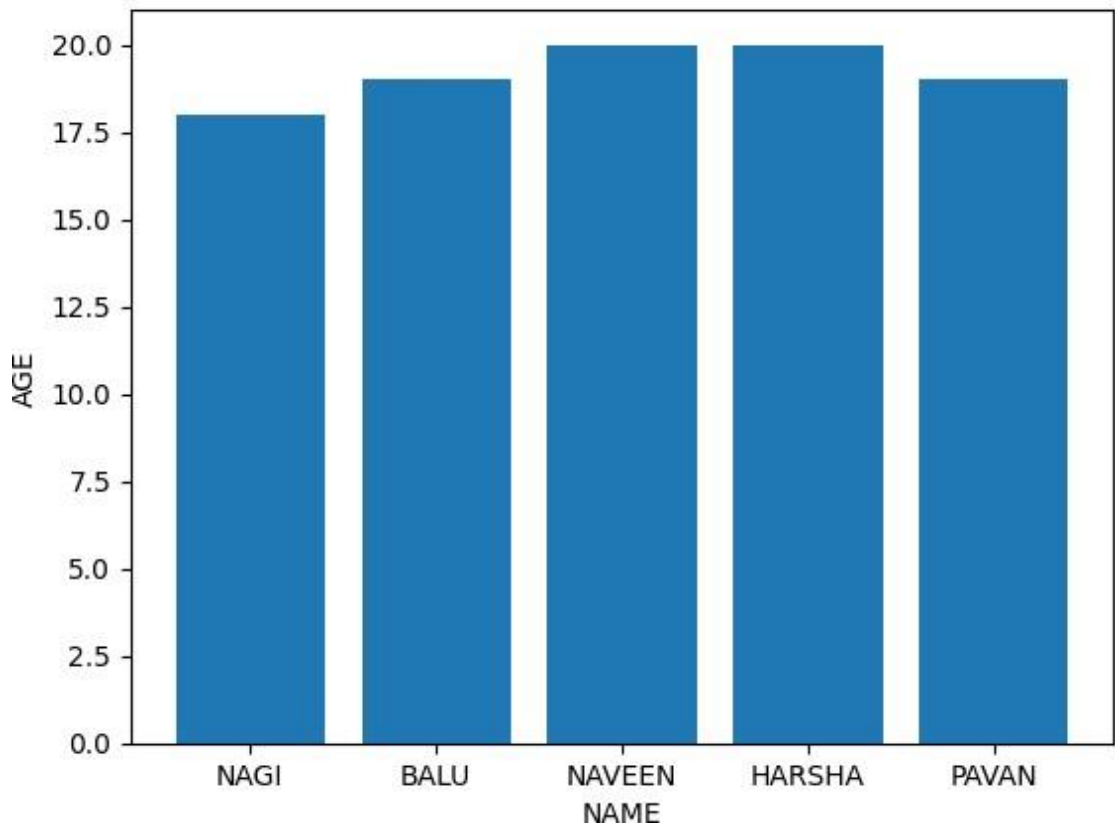
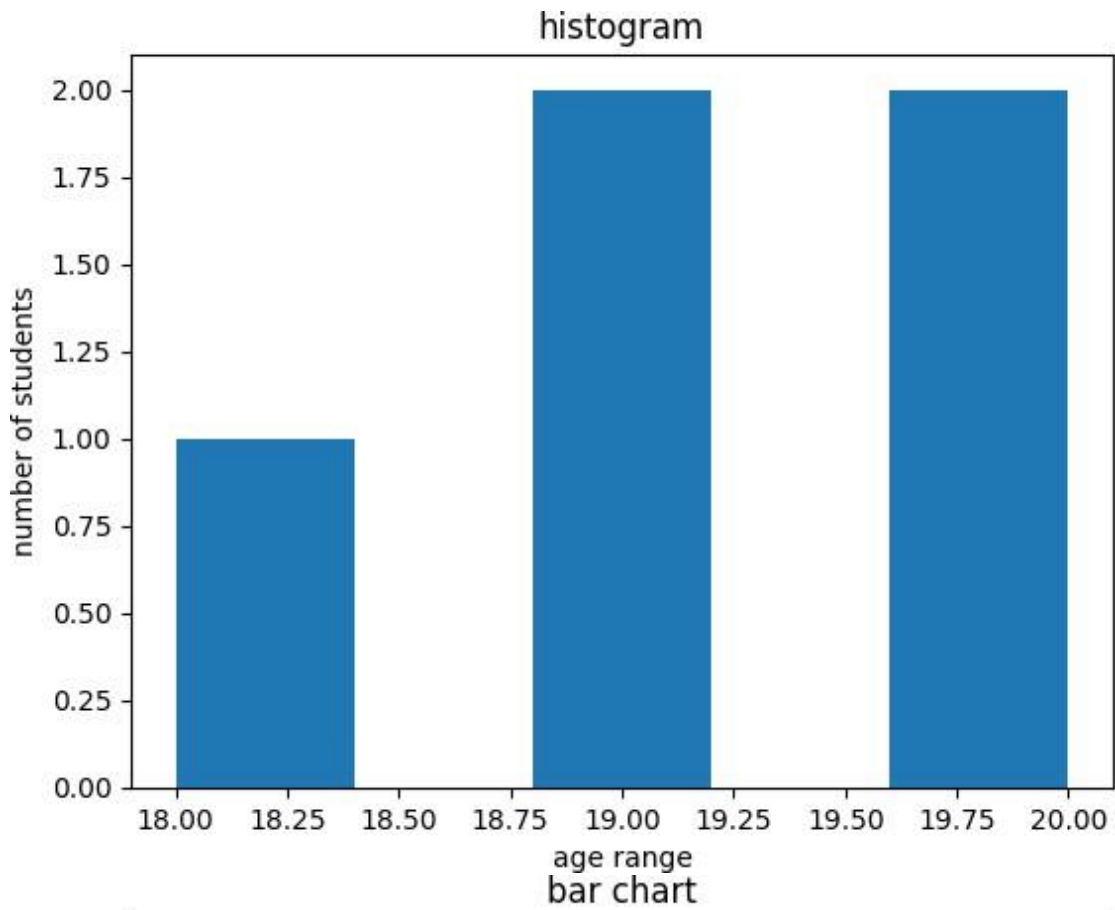


INPUT:

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3  import numpy as np
4
5  df = pd.read_csv('pv.csv')
6
7  plt.scatter(df['NAME'], df['AGE'])
8  plt.xlabel("NAME")
9  plt.ylabel("AGE")
10 plt.title("Scatter Plot")
11 plt.show()
12
13 plt.plot(df['NAME'], df['AGE'], marker='o')
14 plt.xlabel("NAME")
15 plt.ylabel("AGE")
16 plt.title("Line Plot")
17 plt.show()
18
19 plt.hist(df['AGE'], bins=5)
20 plt.xlabel("Age Range")
21 plt.ylabel("Number of Students")
22 plt.title("Histogram")
23 plt.show()
24
25 plt.bar(df['NAME'], df['AGE'])
26 plt.xlabel("NAME")
27 plt.ylabel("AGE")
28 plt.title("Bar Chart")
29 plt.show()
```

OUTPUT:





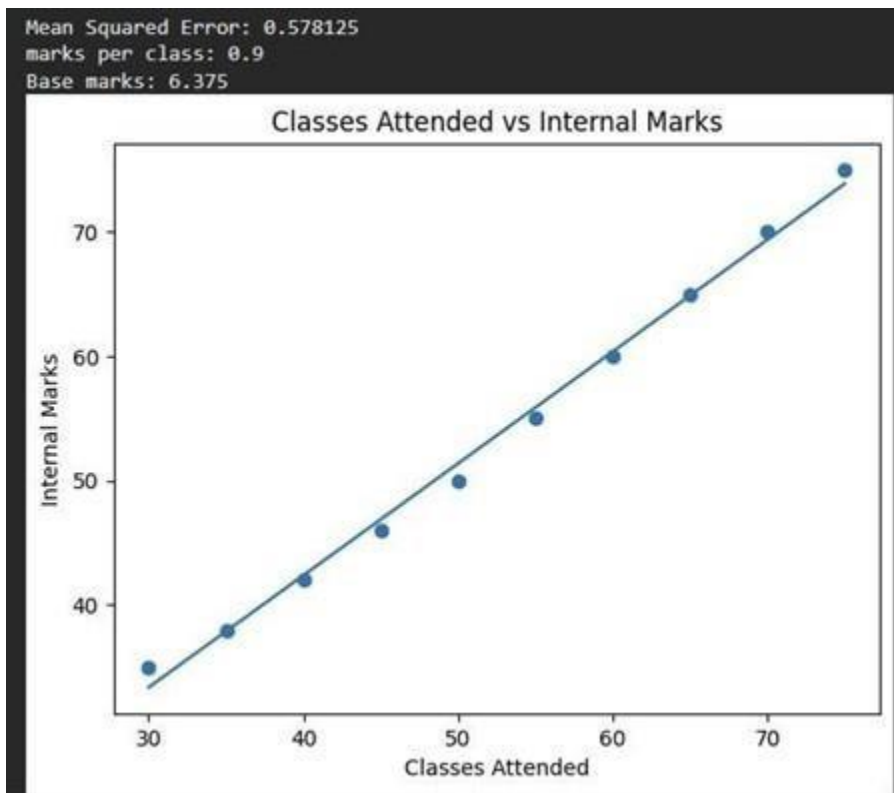
INPUT:

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.model_selection import train_test_split
4 from sklearn.linear_model import LinearRegression
5 from sklearn.metrics import mean_squared_error
6
7 df = pd.read_csv('/content/data.csv')
8
9 X = df[['Classes_Attended']]
10 y = df['Internal_Marks']
11
12 X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.2, random_state=7)
13
14 model = LinearRegression()
15 model.fit(X_train, y_train)
16
17 predictions = model.predict(X_test)
18
19 mse = mean_squared_error(y_test, predictions)
20
21 print("Mean Squared Error:", mse)
22 print("Marks per class:", model.coef_[0])
23 print("Base marks:", model.intercept_)
24
25 plt.scatter(X, y)
26 plt.plot(*args: X, _model_predict(X))
27 plt.xlabel("Classes Attended")
28 plt.ylabel("Internal Marks")
29 plt.title("Classes Attended vs Internal Marks")
30 plt.show()
31

```

OUTPUT:



Result: Basic operations using these libraries were performed successfully.

Ex No:02

Date:12-02-26

Perform Data exploration and preprocessing in Python

Aim:To perform data exploration and preprocessing in

Python.

Program:

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.model_selection import train_test_split
5
6 df = pd.read_csv('/content/data_output.csv')
7 print("---Original raw Data---")
8 print(df)
9 print("\n")
10 print("---missing values count---")
11 print(df.isnull().sum())
12 print("\n")
13 df['Age'] = df['Age'].fillna(df['Age'].mode()[0])
14 df['Salary'] = df['Salary'].fillna(df['Salary'].mode()[0])
15 print("---DATA AFTER CLEANING---")
16 print(df)
17 print("\n")
18 df_encoded = pd.get_dummies(df, columns=['Country'])
19 df_encoded['Purchased'] = df_encoded['Purchased'].map({'Yes': 1, 'No': 0})
20 print("---DATA AFTER ENCODING---")
21 print(df_encoded)
22 print("\n")
23 X = df_encoded.drop(labels='Purchased', axis=1)
24 y = df_encoded['Purchased']
25 scaler = StandardScaler()
26 X_scaled = scaler.fit_transform(X)
27 print("---FINAL PROCESSED DATA (Ready for AI)---")
28 print(pd.DataFrame(X_scaled, columns=X.columns).head())
29

```

OUTPUT:

```

0   India  22.0  40000.0    No
1   USA    25.0  60000.0    Yes
... 2   India  NaN  50000.0    Yes
3   USA    30.0    NaN    No
4   UK     28.0  72000.0    Yes
5   India  35.0  58000.0    Yes

---missing values count---
Country    0
Age        1
Salary     1
Purchased  0
dtype: int64

---DATA AFTER CLEANING---
Country  Age  Salary  Purchased
0   India  22.0  40000.0    No
1   USA    25.0  60000.0    Yes
2   India  28.0  50000.0    Yes
3   USA    30.0  60000.0    No
4   UK     28.0  72000.0    Yes
5   India  35.0  58000.0    Yes

---DATA AFTER ENCODING---
Age  Salary  Purchased  Country_India  Country_UK  Country_USA
0  22.0  40000.0         0         True      False      False
1  25.0  60000.0         1        False      False       True
2  28.0  50000.0         1         True      False      False
3  30.0  60000.0         0        False      False       True
4  28.0  72000.0         1        False       True      False
5  35.0  58000.0         1         True      False      False

---FINAL PROCESSED DATA(Ready for ai)---
Age  Salary  Country_India  Country_UK  Country_USA
0 -1.484615 -1.693214         1.0   -0.447214   -0.707107
1 -0.742307  0.338643        -1.0   -0.447214    1.414214
2  0.000000 -0.677285         1.0   -0.447214   -0.707107
3  0.494872  0.338643        -1.0   -0.447214    1.414214
4  0.000000  1.557757        -1.0    2.236068   -0.707107

```

Result: Data was explored and preprocessed successfully using Python.

Ex No:03

Date:19-02-26

Implement regularised Linear regression

Aim: To implement regularized linear regression to improve model performance and control overfitting.

Program:

```
1 from sklearn import datasets
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LinearRegression, Ridge, Lasso
4 from sklearn.metrics import mean_squared_error
5
6 diabetes = datasets.load_diabetes()
7
8 X_train, X_test, y_train, y_test = train_test_split(
9     *arrays: diabetes.data, diabetes.target, test_size=0.2, random_state=42
10 )
11
12 lr = LinearRegression().fit(X_train, y_train)
13 ridge = Ridge(alpha=1.0).fit(X_train, y_train)
14 lasso = Lasso(alpha=1.0).fit(X_train, y_train)
15
16 print(f"Linear Regression MSE: {mean_squared_error(y_test, lr.predict(X_test)):.2f}")
17 print(f"Ridge Regression MSE: {mean_squared_error(y_test, ridge.predict(X_test)):.2f}")
18 print(f"Lasso Regression MSE: {mean_squared_error(y_test, lasso.predict(X_test)):.2f}")
19
```

OUTPUT:

```
Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 3403.58

Process finished with exit code 0
```

Result: The model was successfully implemented, and regularization helped reduce overfitting and improved prediction accuracy.

Ex No:04

Date:05-02-26

Implement Naive Bayes classifier for dataset stored as CSV file

Aim:

To implement the Naive Bayes classifier using a dataset stored in a CSV file for classification.

Program:


```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.naive_bayes import GaussianNB
4 from sklearn.preprocessing import LabelEncoder
5 from sklearn.metrics import accuracy_score
6 import matplotlib.pyplot as plt
7 data = pd.read_csv('/content/temp_hum_play_data.csv')
8 le = LabelEncoder()
9 data['Play'] = le.fit_transform(data['Play'])
10 X = data[['Temperature', 'Humidity']]
11 y = data['Play']
12 X_train, X_test, y_train, y_test = train_test_split(
13     *arrays: X, y, test_size=0.2, random_state=42
14 )
15 model = GaussianNB()
16 model.fit(X_train, y_train)
17 prediction = model.predict(X_test)
18 all_prediction = model.predict(X)
19 print("Accuracy:", accuracy_score(y_test, prediction))
20 print("\nClass priors:\n", model.class_prior_)
21 print("\nMean (theta):\n", model.theta_)
22 print("\nVariance:\n", model.var_)
23 plt.figure(figsize=(8, 6))
24 plt.scatter(X['Temperature'], X['Humidity'], c=y,
25             cmap='coolwarm', marker='o', label='Actual')
26 plt.scatter(X['Temperature'], X['Humidity'], c=all_prediction,
27             cmap='coolwarm', marker='x', s=100, label='Predicted')
28 plt.xlabel("Temperature")
29 plt.ylabel("Humidity")
30 plt.title("Actual vs Predicted (All Data)")
31 plt.legend()
32 plt.show()
33

```

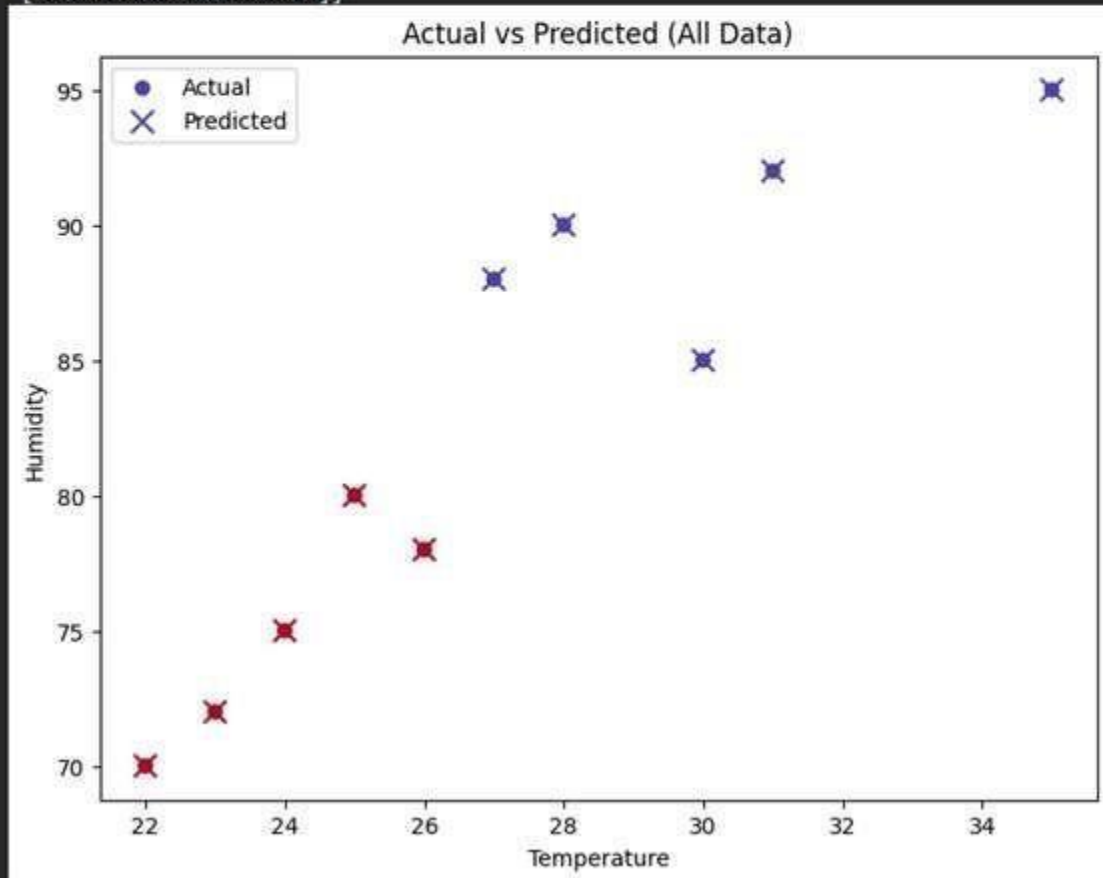
Output:

Accuracy: 1.0

```
*** class priors:  
[0.375 0.625]
```

```
Mean (theta) :  
[[30.66666667 89.33333333]  
 [24.         75.         ]]
```

```
Variance:  
[[10.88888895 17.55555562]  
 [ 2.00000006 13.60000006]]
```



Result:

The Naive Bayes model was successfully implemented, and it effectively classified the data with good accuracy.

Implement regularized logistic regression

Aim:

To implement regularized logistic regression for classification while reducing overfitting.

Program:

```
1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  from sklearn.datasets import load_breast_cancer
5  from sklearn.linear_model import LogisticRegression
6  from sklearn.model_selection import train_test_split
7  from sklearn.preprocessing import StandardScaler
8  from sklearn.metrics import accuracy_score
9  data = load_breast_cancer()
10 X, y = data.data, data.target
11 features = data.feature_names
12 X_train, X_test, y_train, y_test = train_test_split(
13     *arrays: X, y, test_size=0.2, random_state=42
14 )
15 scaler = StandardScaler()
16 X_train = scaler.fit_transform(X_train)
17 X_test = scaler.transform(X_test)
18 normal = LogisticRegression(penalty=None, max_iter=5000).fit(X_train, y_train)
19 normal_acc = accuracy_score(y_test, normal.predict(X_test))
20 ridge = LogisticRegression(penalty='l2', C=1.0, max_iter=5000).fit(X_train, y_train)
21 ridge_acc = accuracy_score(y_test, ridge.predict(X_test))
22 print(f"Normal Accuracy: {normal_acc:.3f}")
23 print(f"Ridge (L2) Accuracy: {ridge_acc:.3f}")
24 print("\nNormal Logistic Coefficients:")
25 print(pd.Series(np.round(normal.coef_[0], decimals=3), index=features))
26 print("\nRidge (L2) Coefficients:")
27 print(pd.Series(np.round(ridge.coef_[0], decimals=3), index=features))
28 plt.plot(*args: normal.coef_[0], label='Normal', color='blue')
29 plt.plot(*args: ridge.coef_[0], label='Ridge (L2)', color='red', marker='o')
30 plt.axhline(y=0, linestyle='--', color='black')
31 plt.legend()
32 plt.title('Coefficient Comparison')
33 plt.show()
```

Output:

Normal Accuracy: 0.939

Ridge (L2) Accuracy: 0.974

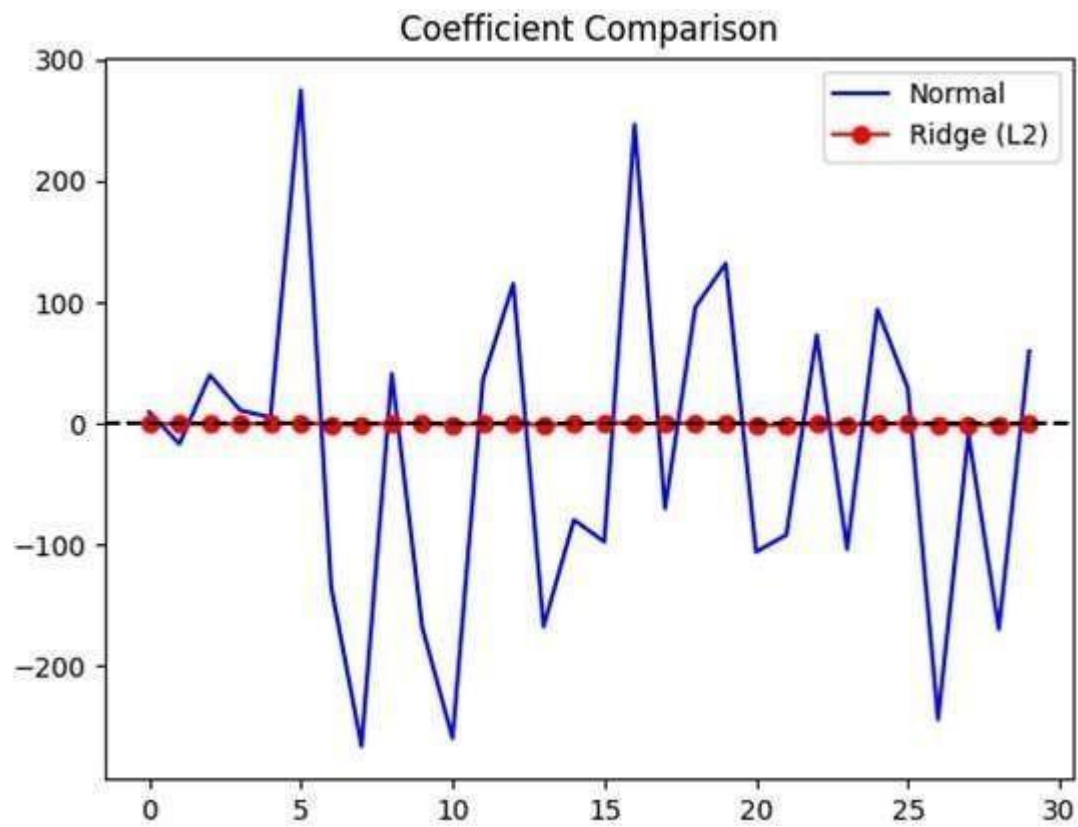
Normal Logistic Coefficients:

mean radius	9.433
mean texture	-16.986
mean perimeter	40.021
mean area	10.890
mean smoothness	5.033
mean compactness	274.940
mean concavity	-134.838
mean concave points	-266.185
mean symmetry	40.997
mean fractal dimension	-168.018
radius error	-259.639
texture error	35.918
perimeter error	115.350
area error	-167.389
smoothness error	-79.540
compactness error	-97.574
concavity error	246.608
concave points error	-70.052
symmetry error	95.954
fractal dimension error	131.939
worst radius	-105.871
worst texture	-91.921
worst perimeter	73.019
worst area	-103.737
worst smoothness	93.975
worst compactness	29.285
worst concavity	-244.060
worst concave points	-8.908
worst symmetry	-169.655

```
worst fractal dimension    39.084  
dtype: float64
```

Ridge (L2) Coefficients:

```
mean radius      -0.432  
mean texture     -0.387  
mean perimeter   -0.393  
mean area        -0.465  
mean smoothness  -0.072  
mean compactness  0.540  
mean concavity    -0.801  
mean concave points -1.120  
mean symmetry     0.236  
mean fractal dimension  0.076  
radius error     -1.268  
texture error     0.189  
perimeter error  -0.611  
area error        -0.907  
smoothness error  -0.313  
compactness error  0.682  
concavity error   0.175  
concave points error -0.311  
symmetry error    0.500  
fractal dimension error  0.616  
worst radius      -0.880  
worst texture     -1.351  
worst perimeter   -0.589  
worst area        -0.842  
worst smoothness  -0.544  
worst compactness  0.016  
worst concavity    -0.943  
worst concave points -0.778  
worst symmetry     -1.208  
worst fractal dimension -0.157  
dtype: float64
```



Result:

The model was successfully implemented, and regularization improved accuracy and prevented overfitting.

Ex

Build

No:06

Date:12-03-26

models using different Ensembling techniques

Aim:

To build models using different ensembling techniques to improve prediction performance

Program:

```
1  from sklearn import datasets
2  from sklearn.model_selection import train_test_split
3  from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
4  from sklearn.svm import SVC
5  from sklearn.linear_model import LogisticRegression
6  from sklearn.metrics import accuracy_score
7
8  iris = datasets.load_iris()
9
10 X_train, X_test, y_train, y_test = train_test_split(
11     *arrays: iris.data, iris.target, test_size=0.2, random_state=42
12 )
13
14 rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
15 ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)
16
17 estimators = [
18     ('rf', RandomForestClassifier()),
19     ('svc', SVC(probability=True))
20 ]
21
22 stack = StackingClassifier(
23     estimators=estimators,
24     final_estimator=LogisticRegression()
25 ).fit(X_train, y_train)
26
27 print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
28 print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
29 print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

Output:

```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

Process finished with exit code 0
```

Ex

Build

Result:

The ensemble models were successfully implemented, and they provided better accuracy compared to individual models.

No:07

Date:12-03-26

models using Decision trees

Aim:

To build a model using Decision Trees for classification or prediction.

Program:

```
1 from sklearn import datasets
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.metrics import accuracy_score
5 iris = datasets.load_iris()
6 X_train, X_test, y_train, y_test = train_test_split(
7     *arrays: iris.data, iris.target, test_size=0.2, random_state=42
8 )
9 dt_clf = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)
10 print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, dt_clf.predict(X_test)):.2f}")
11 print("Feature Importances:", dt_clf.feature_importances_)
```

Output:

```
Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.         0.         0.39426873 0.60573127]

Process finished with exit code 0
```


Ex

Build

Result:

The Decision Tree model was successfully implemented and provided good prediction performance.

No:08

Date:02-03-26

model using SVM with different kernels

Aim:

To build an SVM model using different kernel functions for classification.

Program:

```
1  # Import libraries
2  import pandas as pd
3  from sklearn.model_selection import train_test_split
4  from sklearn.svm import SVC
5  from sklearn.metrics import accuracy_score
6  from sklearn.datasets import load_iris
7  data = load_iris()
8  X = data.data
9  y = data.target
10 X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.2)
11 model_linear = SVC(kernel='linear')
12 model_linear.fit(X_train, y_train)
13 pred_linear = model_linear.predict(X_test)
14 acc_linear = accuracy_score(y_test, pred_linear)
15 print('Linear Kernel Accuracy:', acc_linear)
16 model_poly = SVC(kernel='poly', degree=3)
17 model_poly.fit(X_train, y_train)
18 pred_poly = model_poly.predict(X_test)
19 acc_poly = accuracy_score(y_test, pred_poly)
20 print('Polynomial Kernel Accuracy:', acc_poly)
21 model_rbf = SVC(kernel='rbf')
22 model_rbf.fit(X_train, y_train)
23 pred_rbf = model_rbf.predict(X_test)
24 acc_rbf = accuracy_score(y_test, pred_rbf)
25 print('RBF Kernel Accuracy:', acc_rbf)
```

Output:

Ex

Build

```
Linear Kernel Accuracy: 1.0  
Polynomial Kernel Accuracy: 0.9000000000000007  
RBF Kernel Accuracy: 1.0  
Process finished with exit code 0
```

Result:

The SVM models were successfully implemented with various kernels, and they achieved good classification performance.

Ex

No:09

Date:02-03-26

Implement K-NN algorithm to classify a dataset

Aim: To implement the K-Nearest Neighbors (K-NN) algorithm to classify a dataset
Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
data = pd.read_csv("mobile_price.csv")
X = data.drop("price_range", axis=1)
y = data["price_range"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
*** Accuracy: 1.0
Confusion Matrix:
[[1]]
Classification Report:
      precision    recall  f1-score   support

     2         1.00      1.00      1.00         1
 accuracy         1.00
 macro avg         1.00
weighted avg         1.00
```

Result:

The K-NN model was successfully implemented and it classified the dataset with good accuracy

Ex No:10

Date:02-03-26

Build model to perform Clustering using K-means after applying PCA and determining

Aim:

To build a clustering model using K-Means after applying PCA for dimensionality reduction and determining optimal clusters.

Program:

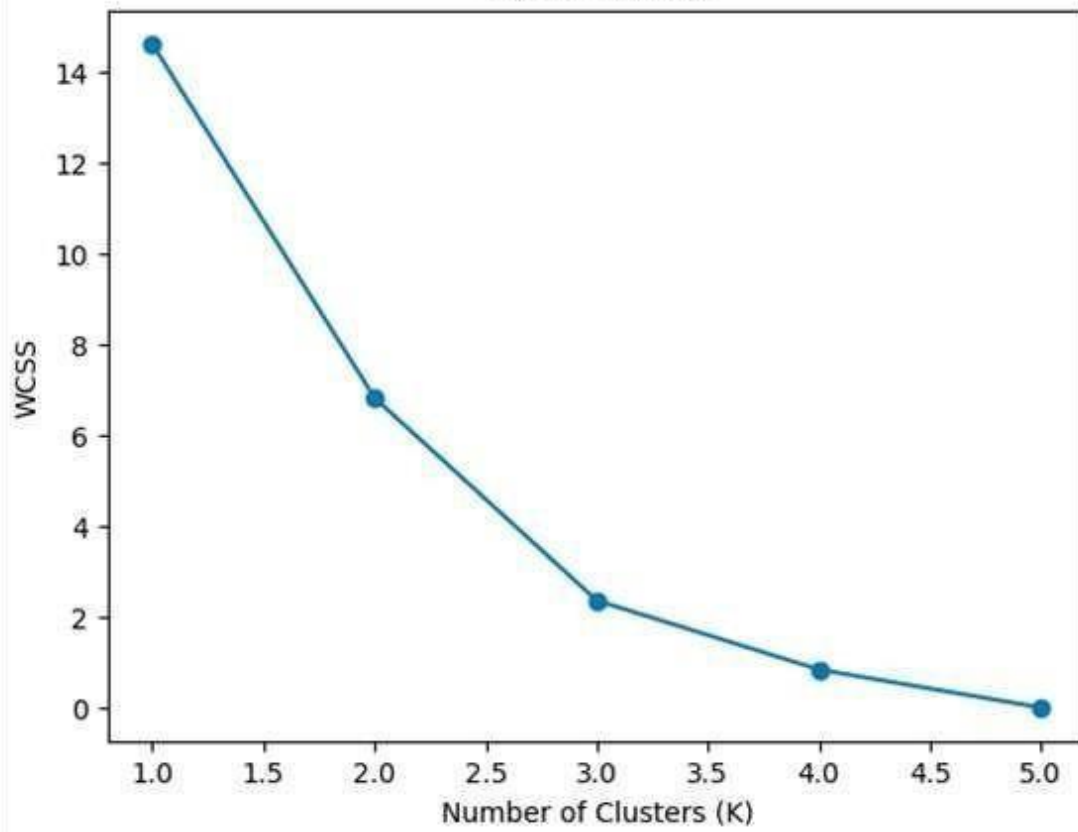
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
data = pd.read_csv("Mall_Customers.csv")
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
wcss = []
max_k = min(10, len(X_pca))
for i in range(1, max_k + 1):
    kmeans = KMeans(n_clusters=i, random_state=0, n_init=10)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.plot(range(1, max_k + 1), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
optimal_k = min(3, len(X_pca))
kmeans = KMeans(n_clusters=optimal_k, random_state=0, n_init=10)
clusters = kmeans.fit_predict(X_pca)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis')
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

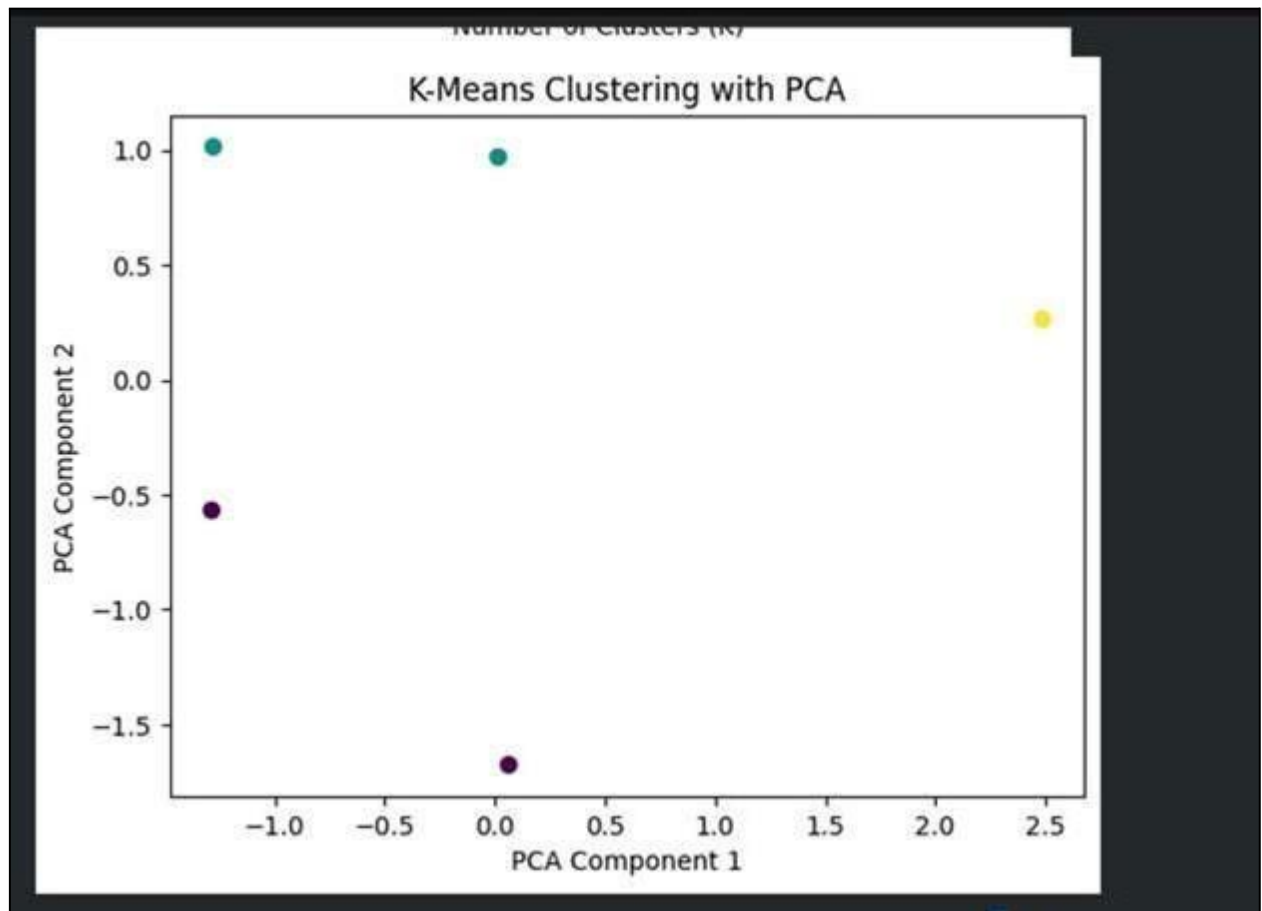
#P.ADITYA
#11249A301

Output:

Explained Variance Ratio: [0.63071118 0.34476153]

Elbow Method





Result:

The K-Means clustering model was successfully implemented after PCA, and the data was grouped effectively into clusters